5COSC0020W DATABASE SYSTEMS – LECTURE 06

SQL Database Querying – Join Queries

Retrieving records from multiple tables in SQL

Dr Chathura Sooriya-Arachchi

c.sooriyaarachchi1@westminster.ac.uk

UNIVERSITY OF WESTMINSTER

Lecture 06 – Outline

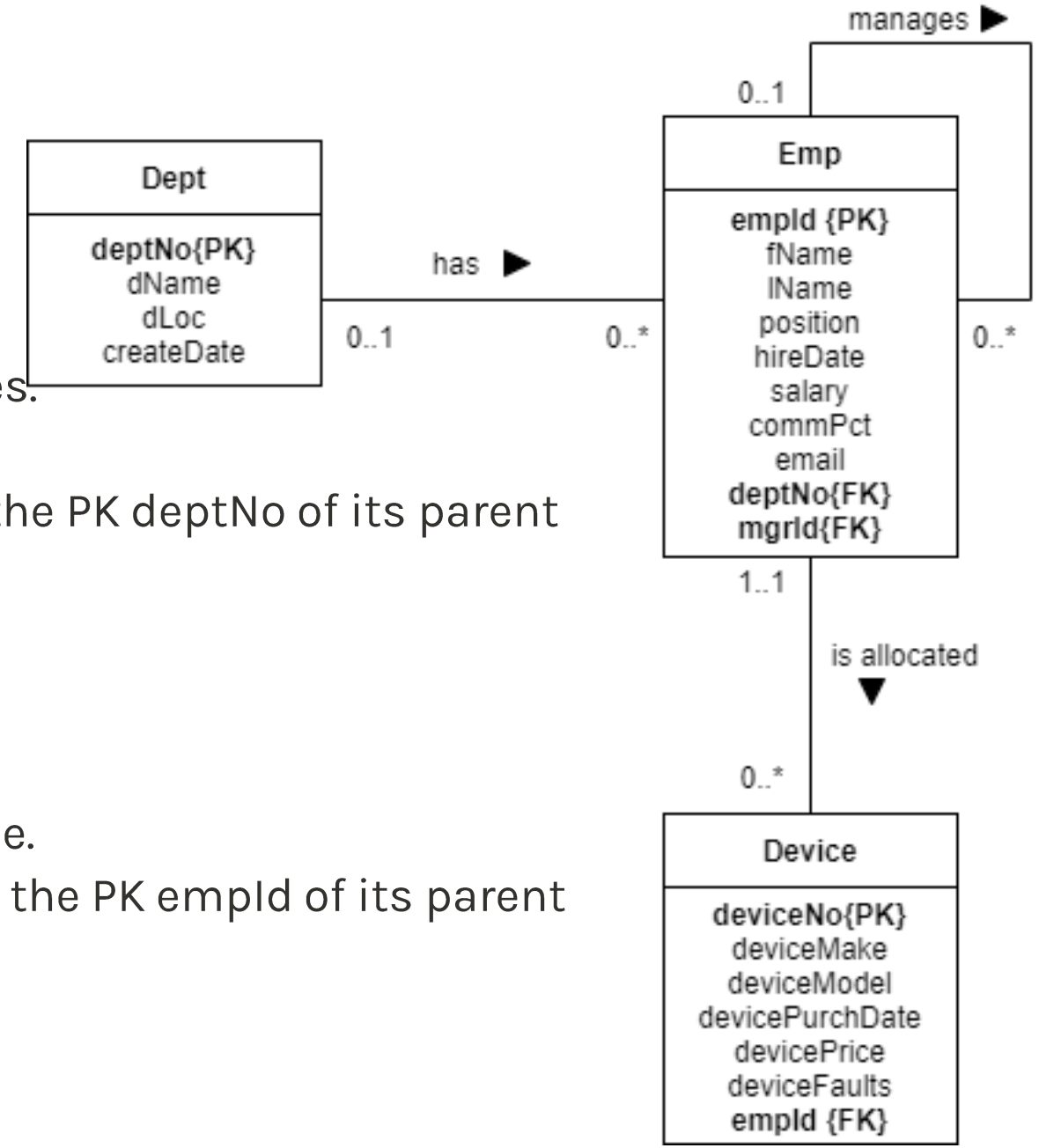
- Recap - SQL to create and populate tables, Simple Queries
- Join statements in SQL to query multiple tables.
- Join statements with ON clause and WHERE clause.
- Join statements with additional conditions.
- Self-join statements.
- Three-way join statements.
- Inner Join statements.
- Left-Outer join statements.
- Right-Outer join statements.
- Full-Outer Join using UNION.

SQL Statements and subsets of SQL

SELECT **INSERT** **UPDATE** **DELETE** **MERGE**	Data Manipulation Language (DML)
CREATE **ALTER** **DROP** **RENAME** **TRUNCATE** **COMMENT**	Data Definition Language (DDL)
GRANT **REVOKE**	Data Control Language (DCL)
COMMIT **ROLLBACK** **SAVEPOINT**	Transaction Control

Logical ERD

- Emp has a recursive relationship.
- Between Dept and Emp
  - One-to-Many relationship optional on both sides.
  - Dept is the Parent table, Emp is the Child table.
  - Child Emp carries a FK deptNo that references the PK deptNo of its parent Dept.
  - Participations are both 0.
- Between Emp and Device
  - One-to-Many relationship optional on one side.
  - Emp is the Parent table, Device is the Child table.
  - Child Device carries a FK empId that references the PK empId of its parent Emp.
  - Participation is 1 on the Emp side.
  - Participation is 1 on the Device side.



Creating Dept table

```
DROP TABLE IF EXISTS Device;
DROP TABLE IF EXISTS Emp;
DROP TABLE IF EXISTS Dept;
```

```
CREATE TABLE Dept
(
    deptNo          INTEGER,
    dName           VARCHAR(20) UNIQUE NOT NULL,
    dLoc            VARCHAR(30) NOT NULL,
    createDate      DATE,
    CONSTRAINT      d_dno_pk PRIMARY KEY (deptNo)
);
```

Creating Emp table

```
CREATE TABLE Emp
(
     empId          INTEGER,
     fName          VARCHAR(50)  NOT NULL,
     lName          VARCHAR(50)  NOT NULL,
     position       VARCHAR(50)  NOT NULL,
     hireDate       DATE                 NOT NULL,
     salary DECIMAL(7,2) NOT NULL,
     commPct        DECIMAL(2,2),
     email          VARCHAR(100) UNIQUE NOT NULL,
     deptNo INTEGER,
     mgrId          INTEGER,
     CONSTRAINT e_eid_pk PRIMARY KEY (empId),
     CONSTRAINT e_dno_fk FOREIGN KEY (deptNo) REFERENCES Dept(deptNo),
     CONSTRAINT e_mid_fk FOREIGN KEY (mgrId) REFERENCES Emp(empId)
);
```

Creating Device table

```
CREATE TABLE Device
(
    deviceNo          INTEGER,
    deviceMake        VARCHAR(50)      NOT NULL,
    deviceModel       VARCHAR(50)      NOT NULL,
    devicePurchDate   DATE             NOT NULL,
    devicePrice       DECIMAL(7,2)     NOT NULL,
    deviceFaults      VARCHAR(200),
    empId             INTEGER          NOT NULL,
    CONSTRAINT d_dno_pk PRIMARY KEY (deviceNo),
    CONSTRAINT d_eid_fk FOREIGN KEY (empId) REFERENCES Emp(empId)
);
```

Populating Dept tables

```
INSERT INTO
Dept (deptNo, dName, dLoc, createDate)
VALUES
(10, 'Database Management', 'New Cav', '2021-03-21'),
(20, 'Systems Design', 'Harrow', '2021-01-06'),
(30, 'IT Development', 'New Cav', '2021-04-25'),
(40, 'Project Management', 'Regents', '2021-02-12'),
(50, 'Systems Testing', 'Marylebone', NULL);
```

Populating Emp tables

```
INSERT INTO
Emp (empId, fName, lName, position, hireDate, salary, commPct, email, deptNo, mgrId)
VALUES
(101, 'Joe', 'Bloggs', 'Project Manager', '2021-03-01', 5200.00, 0.25, 'jb@it.com', 40, NULL),
(102, 'Jim', 'Marts', 'Database Admin', '2021-03-22', 4400.00, NULL, 'jm@it.com', 10, 101),
(103, 'Jen', 'Fonts', 'Python Developer', '2021-04-28', 4800.00, 0.12, 'jf@it.com', 30, 101),
(104, 'Jon', 'Pop', 'Database Architect', '2021-03-23', 4000.00, NULL, 'jp@it.com', 10, 102),
(105, 'Tom', 'Dogs', 'UI Designer', '2021-01-10', 4000.00, NULL, 'td@it.com', 20, 102),
(106, 'Tek', 'Roggs', 'Project Manager', '2021-01-01', 5200.00, 0.13, 'tr@it.com', 40, NULL),
(107, 'Tim', 'Clogs', 'Java Developer', '2021-04-29', 4300.00, 0.15, 'tc@it.com', NULL, 105),
(108, 'Tam', 'Kelps', 'UX Designer', '2021-06-15', 3900.00, NULL, 'tk@it.com', NULL, 106);
```

Populating Device tables

```
INSERT INTO
Device
(deviceNo, deviceMake, deviceModel, devicePurchDate, devicePrice, deviceFaults, empId)
VALUES
(751, 'Pear Phone', 'P Phone 13', '2021-01-12', 976.00, NULL, 106),
(752, 'Pear Tablet', 'P Tablet 04', '2021-01-12', 854.00, NULL, 106),
(753, 'Space Laptop', 'S PC 11', '2021-01-13', 1231.00, 'Scratched', 106),
(754, 'Space Phone', 'S Phone 11', '2021-03-23', 887.00, NULL, 101),
(755, 'Pear Tablet', 'P Tablet 05', '2021-03-23', 1045.00, NULL, 101),
(756, 'Pear Phone', 'P Phone 13', '2021-04-02', 976.00, NULL, 102),
(757, 'Pear Phone', 'P Phone 13', '2021-04-01', 976.00, NULL, 104),
(758, 'Pear Phone', 'P Phone 13', '2021-02-12', 976.00, 'Slow', 105),
(759, 'Space Laptop', 'S PC 54', '2021-02-13', 1143.00, NULL, 105),
(760, 'Space Tablet', 'S Tab 2312', '2021-02-15', 998.00, NULL, 105),
(761, 'Move Phone', 'M Phone 32', '2021-05-07', 1241.00, 'Crashes', 103),
(762, 'Move Laptop', 'M Laptop 3223', '2021-05-08', 1678.00, NULL, 103);
```

Recap from Simple SQL Queries

- Display the last name, position and salary for all employees whose department number is 10, 30 or 40
  and whose salary is not equal to 4000, 4400 or 4300.

Recap from Simple SQL Queries contd.

- Display the last name, position and salary for all employees whose department number is 10, 30 or 40 and whose salary is not equal to 4000, 4400 or 4300.

```
SELECT lName, position, salary
FROM   Emp
WHERE  deptNo IN (10, 30, 40)
AND    salary NOT IN (4000, 4400, 4300);
```

lName	position	salary
Fonts	Python Developer	4800.00
Bloggs	Project Manager	5200.00
Roggs	Project Manager	5200.00

Recap from Simple SQL Queries contd.

- Display the last name, position, salary and hire date for all employees
  for whom either their position is ‘Database Admin’ or ‘UI Designer’
  and their salary is less than 4500,
  or they were hired before 1st April 2021
  and their last name has the letter ‘o’ as the one letter before last.

Recap from Simple SQL Queries contd.

- Display the last name, position, salary and hire date for all employees for whom either their position is ‘Database Admin’ or ‘UI Designer’ and their salary is less than 4500, or they were hired before 1st April 2021 and their last name has the letter ‘o’ as the one letter before last.

```
SELECT lName, position, salary, hireDate
FROM   Emp
WHERE  (position IN ('Database Admin', 'UI Designer')
AND salary < 4500)
OR  (hireDate < '2021-04-01’
AND lName LIKE '%o_');
```

lName	position	salary	hireDate
Marts	Database Admin	4400.00	2021-03-22
Pop	Database Architect	4000.00	2021-03-23
Dogs	UI Designer	4000.00	2021-01-10

Why Joins? Retrieving related data from multiple tables

WHY WE NEED JOINS

✓ Data is split across tables
- Dept holds departments; Emp holds employees
- Each fact stored once: less repeated data

✓ Tables are linked by keys
- Child carries a FK that points to the Parent PK
- e.g. Emp.deptNo → Dept.deptNo

✓ One table rarely answers the question
- “Which department does Mr Pop work in?”
- Needs data from both Emp and Dept

✓ Some rows have no match
- Staff with no department (Clogs, Kelps)
- Departments with no staff (Systems Testing)

WHAT A JOIN GIVES US

✓ Combining row data
- Employee + Department information in one result

✓ Record matching
- Link related rows using common keys
- Parent.PK = Child.FK

✓ Multiple table columns
- Show related records across 2, 3 or more tables
- Dept → Emp → Device

✓ You choose which rows to keep
- JOIN: matching rows only
- LEFT / RIGHT / FULL (through UNION): also keep unmatched rows

Join statement using ON clause (without prefixes)

- Join statement to retrieve data from 2 tables.
- It retrieves records in parent table with matching records in child table.
- Records in parent table without matching records in child table not added.
- Records in child table without matching records in parent table not added.

```
SELECT dName, dLoc, lName, position, email
FROM Dept JOIN Emp
ON Dept.deptNo = Emp.deptNo;
```

Parent.PK = Child.FK

Join statement using ON clause (with prefixes)

- With prefixes (preferred).

```
SELECT Dept.dName, Dept.dLoc, Emp.lName, Emp.email
FROM Dept JOIN Emp
ON Dept.deptNo = Emp.deptNo;
```

- Prefix required if retrieving the Parent's PK/Child's FK as ambiguous.

```
SELECT Dept.deptNo, Dept.dName, Emp.lName,Emp.email
FROM Dept JOIN Emp
ON Dept.deptNo = Emp.deptNo;
```

Parent.PK = Child.FK

Join statement using ON clause (with prefixes & aliases)

- With prefixes and with aliases (best one!)

```
SELECT D.dName, D.dLoc, E.lName, E.email
FROM Dept D JOIN Emp E
ON D.deptNo = E.deptNo;
```

Parent.PK = Child.FK

dName	dLoc	lName	email
Database Management	New Cav	Marts	jm@it.com
Database Management	New Cav	Pop	jp@it.com
Systems Design	Harrow	Dogs	td@it.com
IT Development	New Cav	Fonts	jf@it.com
Project Management	Regents	Bloggs	jb@it.com
Project Management	Regents	Roggs	tr@it.com

Join statement using WHERE clause (with prefixes & aliases)

- With prefixes and with aliases

```
SELECT D.dName, D.dLoc, E.lName, E.email
FROM Dept D, Emp E
WHERE D.deptNo = E.deptNo;
```

Parent.PK = Child.FK

dName	dLoc	lName	email
Database Management	New Cav	Marts	jm@it.com
Database Management	New Cav	Pop	jp@it.com
Systems Design	Harrow	Dogs	td@it.com
IT Development	New Cav	Fonts	jf@it.com
Project Management	Regents	Bloggs	jb@it.com
Project Management	Regents	Roggs	tr@it.com

Join statement using ON clause with additional condition

- Additional condition to restrict retrieved records

```
SELECT D.dName, D.dLoc, E.lName, E.email
FROM Dept D JOIN Emp E
ON D.deptNo = E.deptNo
AND D.dName LIKE '%Database%';
```

Parent.PK = Child.FK

dName	dLoc	lName	email
Database Management	New Cav	Marts	jm@it.com
Database Management	New Cav	Pop	jp@it.com

Join condition using WHERE clause with additional condition

- Additional condition to restrict retrieved records

```
SELECT D.dName, D.dLoc, E.lName, E.email
FROM Dept D, Emp E
WHERE D.deptNo = E.deptNo
AND D.dName LIKE '%Database%';
```

Parent.PK = Child.FK

dName	dLoc	lName	email
Database Management	New Cav	Marts	jm@it.com
Database Management	New Cav	Pop	jp@it.com

Join condition using ON clause with 2 additional conditions

- Use brackets to separate conditions

```
SELECT D.dName, D.dLoc, E.lName, E.email
FROM Dept D JOIN Emp E
ON D.deptNo = E.deptNo
AND (D.dName LIKE '%Database%'
OR E.salary < 4700);
```

Parent.PK = Child.FK

dName	dLoc	lName	email
Database Management	New Cav	Marts	jm@it.com
Database Management	New Cav	Pop	jp@it.com
Systems Design	Harrow	Dogs	td@it.com

Self-Join

- Join statement to retrieve data from the same table treating as two separate instances.

- *Create a query*
  to display the id, last name and first name of each manager,
  together with the id, last name and first name of each employee they manage.

Self-Join contd.

- Aliases are required to distinguish between the uses of the same table!
- *Create a query to display the id, last name and first name of each manager, together with the id, last name and first name of each employee they manage.*

```
SELECT M.empId, M.lName, M.fName,
       E.empId, E.lName, E.fName
FROM Emp M JOIN Emp E
ON M.empId = E.mgrId;
```

Parent.PK = Child.FK

Employee under the Manager

Employee under the Manager

empId	lName	fName	empId	lName	fName
101	Bloggs	Joe	102	Marts	Jim
101	Bloggs	Joe	103	Fonts	Jen
102	Marts	Jim	104	Pop	Jon
102	Marts	Jim	105	Dogs	Tom
105	Dogs	Tom	107	Clogs	Tim
106	Roggs	Tek	108	Kelps	Tam

Three-way join statement

- Join statement to retrieve data from 3 tables

- *Create a query to display the department name, location and creation date, the employee last name, first name and email, and the device make, model and faults for all devices allocated to employees who work in a department.*

Three-way join statement

- Join statement to retrieve data from 3 tables
- *Create a query to display the department name, location and creation date, the employee last name, first name and email, and the device make, model and faults for all devices allocated to employees who work in a department.*

```
SELECT D.dName, D.dLoc, D.createDate,
E.lName, E.fName, E.email,
DV.deviceMake, DV.deviceModel, DV.deviceFaults
FROM Dept D
JOIN Emp E          ON D.deptNo = E.deptNo
JOIN Device DV      ON E.empId = DV.empId;
```

Parent.PK = Child.FK

Three-way join statement contd.

- *Create a query to display the department name, location and creation date, the employee last name, first name and email, and the device make, model and faults for all devices allocated to employees who work in a department.*

dName	dLoc	createDate	lName	fName	email	deviceMake	deviceModel	deviceFaults
Database Management	New Cav	2021-03-21	Marts	Jim	jm@it.com	Pear Phone	P Phone 13	*NULL*
Database Management	New Cav	2021-03-21	Pop	Jon	jp@it.com	Pear Phone	P Phone 13	*NULL*
Systems Design	Harrow	2021-01-06	Dogs	Tom	td@it.com	Pear Phone	P Phone 13	Slow
Systems Design	Harrow	2021-01-06	Dogs	Tom	td@it.com	Space Laptop	S PC 54	*NULL*
Systems Design	Harrow	2021-01-06	Dogs	Tom	td@it.com	Space Tablet	S Tab 2312	*NULL*
IT Development	New Cav	2021-04-25	Fonts	Jen	jf@it.com	Move Phone	M Phone 32	Crashes
IT Development	New Cav	2021-04-25	Fonts	Jen	jf@it.com	Move Laptop	M Laptop 3223	*NULL*
Project Management	Regents	2021-02-12	Bloggs	Joe	jb@it.com	Space Phone	S Phone 11	*NULL*
Project Management	Regents	2021-02-12	Bloggs	Joe	jb@it.com	Pear Tablet	P Tablet 05	*NULL*
Project Management	Regents	2021-02-12	Roggs	Tek	tr@it.com	Pear Phone	P Phone 13	*NULL*
Project Management	Regents	2021-02-12	Roggs	Tek	tr@it.com	Pear Tablet	P Tablet 04	*NULL*
Project Management	Regents	2021-02-12	Roggs	Tek	tr@it.com	Space Laptop	S PC 11	Scratched

Inner-Join statement

- Join statement retrieves records with matching values in both tables!
- Sometimes referred to as inner-join

```
SELECT D.dName, D.dLoc, E.lName, E.email
FROM Dept D JOIN Emp E
ON D.deptNo = E.deptNo;
```

Parent.PK = Child.FK

But what about Employees who are not in a department, and Departments that have no Employees???

dName	dLoc	lName	email
Database Management	New Cav	Marts	jm@it.com
Database Management	New Cav	Pop	jp@it.com
Systems Design	Harrow	Dogs	td@it.com
IT Development	New Cav	Fonts	jf@it.com
Project Management	Regents	Bloggs	jb@it.com
Project Management	Regents	Roggs	tr@it.com

Left-Outer Join

- Left-Outer Join retrieves records with matching values in both tables PLUS records from table on the LEFT without matching records on the table on the RIGHT (of the JOIN clause).
- It puts NULL values in the fields without matching values.

*Create a query to display the department name, location, and the last name and email of employees, for **all** departments, including departments that have no employees.*

Left-Outer Join contd.

- *Create a query to display the department name, location, and the last name and email of employees, for all departments, including departments that have no employees.*

```
SELECT D.dName, D.dLoc, E.lName, E.email
FROM Dept D LEFT OUTER JOIN Emp E
ON D.deptNo = E.deptNo;
```

Parent.PK = Child.FK

dName	dLoc	lName	email
Database Management	New Cav	Marts	jm@it.com
Database Management	New Cav	Pop	jp@it.com
Systems Design	Harrow	Dogs	td@it.com
IT Development	New Cav	Fonts	jf@it.com
Project Management	Regents	Bloggs	jb@it.com
Project Management	Regents	Roggs	tr@it.com
Systems Testing	Marylebone	*NULL*	*NULL*

Right-Outer Join condition using ON clause

- Right-Outer Join retrieves records with matching values in both tables PLUS records from table on the RIGHT without matching records on the table on the LEFT (of the JOIN clause).
- It puts NULL values in the fields without matching values.

- *Create a query to display the department name, location, and the last name and email of employees, for all employees, including employees who do not belong to a department.*

```
SELECT D.dName, D.dLoc, E.lName, E.email
FROM Dept D RIGHT OUTER JOIN Emp E
ON D.deptNo = E.deptNo;
```

Parent.PK = Child.FK

Right-Outer Join condition using ON clause contd.

- *Create a query to display the department name, location, and the last name and email of employees, for all employees, including employees who do not belong to a department.*

Dept.deptNo	Emp.deptNo	dName	dLoc	lName	email
40	40	Project Management	Regents	Bloggs	jb@it.com
10	10	Database Management	New Cav	Marts	jm@it.com
30	30	IT Development	New Cav	Fonts	jf@it.com
10	10	Database Management	New Cav	Pop	jp@it.com
20	20	Systems Design	Harrow	Dogs	td@it.com
40	40	Project Management	Regents	Roggs	tr@it.com
NULL	*NULL*	*NULL*	*NULL*	Clogs	tc@it.com
NULL	*NULL*	*NULL*	*NULL*	Kelps	tk@it.com

Full-Outer Join

- Full-Outer Join retrieves records with matching values in both tables
  + Records from table on the RIGHT without matching records on the table on the LEFT (of the JOIN clause)
  +Records from table on the LEFT without matching records on the table on the RIGHT (of the JOIN clause)

- It puts NULL values in the fields without matching values.

- Not supported by MySQL and needs to be written with UNION.

Full-Outer Join with UNION

- *Create a query to display the department name, location, and the last name and email of employees, for all departments and all employees, including those with no match on either side.*

```
SELECT D.dName, D.dLoc, E.lName, E.email
FROM Dept D LEFT OUTER JOIN Emp E
ON D.deptNo = E.deptNo
UNION
SELECT D.dName, D.dLoc, E.lName, E.email
FROM Dept D RIGHT OUTER JOIN Emp E
ON D.deptNo = E.deptNo;
```

Parent.PK = Child.FK

`UNION ALL` can be used instead of `UNION` to keep duplicate records.

Full-Outer Join with UNION contd.

- *Create a query to display the department name, location, and the last name and email of employees, for all departments and all employees, including those with no match on either side.*

Dept.deptNo	Emp.deptNo	dName	dLoc	lName	email
10	10	Database Management	New Cav	Marts	jm@it.com
10	10	Database Management	New Cav	Pop	jp@it.com
20	20	Systems Design	Harrow	Dogs	td@it.com
30	30	IT Development	New Cav	Fonts	jf@it.com
40	40	Project Management	Regents	Bloggs	jb@it.com
40	40	Project Management	Regents	Roggs	tr@it.com
50	*NULL*	Systems Testing	Marylebone	*NULL*	*NULL*
NULL	*NULL*	*NULL*	*NULL*	Clogs	tc@it.com
NULL	*NULL*	*NULL*	*NULL*	Kelps	tk@it.com

References and Essential Readings

- Module Reading List: https://readinglist.westminster.ac.uk/leganto/nui/lists/9326079090003711?auth=SAML

- Connolly, T. & Begg, C. E. (2015). Database systems: a practical approach to design, implementation and management. 6th Edition (Global Edition). Pearson Education.

- Elmasri, R. & Navathe, S. (2017). Fundamentals of Database Systems. 7th Edition (Global Edition). Pearson Education.